ITUPublications

International Telecommunication Union

Recommendation ITU-T SG17-TD990

Provisional identifier: X.icd-schemas
(11/2023)

Vendor agnostic Security Data Schemas for Integrated Cyber Defense Solutions

Recommendation ITU-T SG17-TD990

Vendor agnostic Security Data Schemas for Integrated Cyber Defense Solutions

Abstract

Keywords

Data, schemas, security.

FOREWORD

The International Telecommunication Union (ITU) is the United Nations specialized agency in the field of telecommunications , information and communication technologies (ICTs). The ITU Telecommunication Standardization Sector (ITU-T) is a permanent organ of ITU. ITU-T is responsible for studying technical, operating and tariff questions and issuing Recommendations on them with a view to standardizing telecommunications on a worldwide basis.

The World Telecommunication Standardization Assembly (WTSA), which meets every four years, establishes the topics for study by the ITU T study groups which, in turn, produce Recommendations on these topics.

The approval of ITU-T Recommendations is covered by the procedure laid down in WTSA Resolution 1.

In some areas of information technology which fall within ITU-T's purview, the necessary standards are prepared on a collaborative basis with ISO and IEC.

NOTE

In this Recommendation, the expression "Administration" is used for conciseness to indicate both a telecommunication administration and a recognized operating agency.

Compliance with this Recommendation is voluntary. However, the Recommendation may contain certain mandatory provisions (to ensure, e.g., interoperability or applicability) and compliance with the Recommendation is achieved when all of these mandatory provisions are met. The words "shall" or some other obligatory language such as "must" and the negative equivalents are used to express requirements. The use of such words does not suggest that compliance with the Recommendation is required of any party.

INTELLECTUAL PROPERTY RIGHTS

ITU draws attention to the possibility that the practice or implementation of this Recommendation may involve the use of a claimed Intellectual Property Right. ITU takes no position concerning the evidence, validity or applicability of claimed Intellectual Property Rights, whether asserted by ITU members or others outside of the Recommendation development process.

As of the date of approval of this Recommendation, ITU had not received notice of intellectual property, protected by patents, which may be required to implement this Recommendation. However, implementers are cautioned that this may not represent the latest information and are therefore strongly urged to consult the TSB patent database at http://www.itu.int/ITU-T/ipr/.

Table of Contents

Page

Draft new Recommendation ITU-T SG17-TD990

Vendor agnostic Security Data Schemas for Integrated Cyber Defense Solutions

1. Scope

This scope of this recommendation is the definition of the vendor agnostic security data schemas that products may use to either produce security data or consume security data in the context of an Integrated Cyber Defense (ICD) solution.

2. References

None.

3. Definitions

3.1. Terms defined elsewhere

None.

3.2. Terms defined in this recommendation

This Recommendation defines the following terms:

3.2.1. integrated cyber defense: an initiative for interoperability of security capabilities (products, technologies, organizations, services, etc.).

4. Abbreviations and acronyms

This Recommendation uses the following abbreviations and acronyms:

CDC	Cyber Defence Centre (X.1060)
EoC	Evidence of Compromise
ICD	Integrated Cyber Defense
JSON	JavaScript ObjectNotation
SIEM	Security Information and Event Management
SOC	Security Operation Center

5. Conventions

In this Recommendation:

The phrase "is required" indicates a requirement that must be strictly followed and from which no deviation is permitted if conformance to this document is to be claimed.

The phrase "is recommended" indicates a requirement that is recommended, but which is not absolutely required. Thus, this requirement need not be present to claim conformance.

In this Recommendation, the keywords "shall" and "should" may sometimes appear, in which case they are to be interpreted as "is required to" and "is recommended" respectively.

6. Introduction

An Integrated Cyber Defense (ICD) is an initiative to provide interoperability for security products, technologies and services.

While ICD adopts open standards, it proposes a new standard for security event telemetry.

The ICD Schema is focused on the representation of security activity rather than its transport.

6.1. Benefits of ICD Schema

Most governments and corporations that acquire security products, technologies and services will do so from multiple vendors. Without a common baseline taxonomy, the cost and complexity of integrating is extremely high.

While today, many government agencies and corporations are prepared to do the work of normalizing and integrating for their SIEM and data lakes, the result is often a shallow or minimum implementation that is incompatible with peer organizations.

The vendor agnostic ICD Schema, if adopted, is rich enough to express more interesting and important activity, but also allows for a community of organizations that share the same objectives to shorten the time it takes to analyze data, and collaborate where possible against common threat actors.

6.2. Introduction to the Framework and Schema

This document describes the Framework and Schema and its taxonomy, including the core cybersecurity event schema built with the framework.

The framework is made up of a set of data types and objects, an attribute dictionary, and the taxonomy.

It is not restricted to the cybersecurity domain nor to events, however the initial focus of the framework has been a schema for cybersecurity events. The core schema is intended to be agnostic to implementations. The schema framework definition files and the resulting normative schema are written as JSON.

7. Personas

There are four personas that are users of the framework and the schema built with the framework:

– the author persona is who creates or extends the schema,
– the producer persona is who generates events natively into the schema, or via a translation from another schema,
– the mapper persona is who translates or creates events from another source to the schema,
– the analyst persona is the end user who searches the data, writes rules or analytics against the schema, or creates reports from the schema.

NOTE –

The analyst may also be considered the consumer persona.

For example, a vendor may write a translation from some external source format into the schema but also extend the schema to accommodate source specific attributes or operations. The vendor is operating as both the mapper and author personas. A CDC or SOC analyst that collects the data in a SIEM system writes rules against the events and searches events during investigation. The CDC or SOC analyst is operating as the analyst persona. Finally, a vendor that emits events natively in the schema's form, even if translated, is a data producer.

8. Taxonomies

There are 5 fundamental constructs of the taxonomy:

a) Data Types, Attributes and Arrays
b) Event Classes
c) Categories
d) Profiles
e) Extensions

The scalar data types are defined on top of primitive data types such as strings, integers, floating point numbers and booleans. Examples of scalar data types are Timestamp, IP Address, MAC Address, and User Name.

An attribute is a unique identifier name for a specific validatable data type, either scalar or complex.

Complex data types are termed objects. An object is a collection of contextually related attributes, usually representing an entity, and may include other objects. Each object is also a data type. Examples of object data types are Process, Device, User, Malware and File.

Arrays support any of the data types.

Most scalar data types have constraints on their valid values or ranges, for example Enum integer types are constrained to a specific set of integer values. Enum integer typed attributes are an important part of the framework constructs and used in place of strings where possible to ensure consistency.

Complex data types, or objects, can also be validated based on their particular structure and attribute requirements. Attribute requirements are discussed in a subsequent section.

The attribute dictionary of all available attributes, and their types are the building blocks of the framework. Event classes are particular sets of attributes from the dictionary.

Events are represented by event classes which structure a set of attributes that attempt to describe the semantics of the event in detail. An individual event is an instance of an event class. Event classes have schema-unique IDs. Individual events may have globally unique IDs.

Each event class is grouped by category, and has a unique category_uid attribute value which is the category identifier. Categories also have friendly name captions, such as System Activity, Network Activity, Findings, etc. Event classes are grouped into categories for a number of purposes: a container for a particular event domain, documentation convenience and search, reporting, storage partitioning or access control to name a few.

Profiles overlay additional related attributes into event classes and objects allowing for cross-category event class augmentation and filtering. Event classes register for profiles that when optionally applied, can be mixed into event classes and objects, by a producer or mapper. For example, System Activity event classes may also include attributes for malware detection or vulnerability information when an endpoint security product is the data source. Network Activity event classes from a host computer may carry the device, process and user associated with the activity. A Security Control profile or Host profile can be applied in these cases, respectively.

Finally, extensions allow the schema to be extended using the framework without modification of the core schema. New attributes, objects, event classes, categories and profiles are all available to extensions. Existing profiles can be applied to extensions, and new extension profiles can be applied to core event classes and objects as well as to other extensions.

9. Data types, attributes, objects

Attributes and the dictionary are the building blocks of a schema. This section discusses the schema attribute conventions, requirements, groupings, constraints, and some of the special attributes used in the core cybersecurity schema.

In general, an attribute from the dictionary has the same meaning everywhere it is used in a schema. Some attributes can have a meaning that is overloaded depending on the event class context where they are used. In these cases the description of the attribute will be generic and include a 'see specific usage' instruction to override its description within the event class context rather than in the dictionary.

9.1. Conventions

The scheme adheres to naming conventions in order to more easily identify attributes with similar semantics. These conventions take the form of standard suffixes and prefixes. The standard suffixes are:

`_id`, `_ids`, `_uid`, `_uuid`, `_ip`, `_name`, `_info`, `_detail`, `_time`, `_dt`, `_process`, `_ver`

9.1.1. Arrays

Attribute names used for arrays end with s. For example `category_ids`. A MITRE ATT&CKTM array is named attacks.

9.1.2. Unique IDs

Attribute names for classification values that are unique within the schema end with _uid. Schema classification attributes that have the _uid suffix are integers, preset by the schema definition (i.e. they must be populated as defined by the schema).

Certain schema-unique attributes that also have a friendly name or caption have the same prefix but by convention use the _name suffix. For example, class_uid and class_name, or category_uid and category_name.

Other attributes with the _uid suffix convention may be strings or integers, depending on their purpose, although the majority are strings.

A uid core attribute is used wherever a producer or mapper populates an identifier for an entity object. Entity objects also have a corresponding name attribute by convention. Both are of type string (string_t).

Attribute names for values that are globally unique end with _uuid. They do not have friendly names. For example GUIDs.

9.1.3. Enum Attributes

Attributes that are of an Enum integer type end with _id. Enum constant identifiers are integers from a defined set where each has a friendly name label. Arrays of enum attributes end with _ids.

By convention, every Enum type has two common values with integer value 0 for Unknown and 99 for Other.

If a source event has missing values that are required by the event class for that event, an Unknown value should be set for Enum types which is also the default.

If a mapped event attribute does not have a defined enumeration value corresponding to a value of the event, Other is used which indicates that a sibling string attribute is populated with the custom attribute value. The sibling string attribute has the same name, minus the suffix. For example, activity_id and activity, or severity_id and severity.

Sibling string attributes are optional, but if the enum value is Other (99) then the sibling string must be populated with the custom label (i.e. not "Other").

For all defined enumeration integer values, including Unknown, the enum label text for the item may populate the sibling string attribute. That is, both the integer value and the string attribute are set. If

the Enum attribute is required, then both the integer attribute and the sibling string attribute should be populated. Attribute requirements are discussed in a subsequent section.

9.2. Attribute Requirement Flags

Attributes in the context of an event class have a requirement flag, that depends on the semantics of the event class. Attributes themselves do not have a requirement flag, only within the context of event classes.6

The requirement flags are:
– Required
– Recommended
– Optional

Event classes are designed so that the most essential attributes are required, to give enough meaning and context to the information reported by the data source. If an attribute is required, then a consumer of the event can count on the attribute being present, and its value populated. If a required attribute cannot be populated for a particular event class, a default value is defined by the event class, usually Unknown.7

Recommended attributes should be populated but cannot be in all cases and unlike required attributes are not subject to validation. They do not have default values. Optional attributes may be populated to add context and when data sources emit richer information.

Some event classes may specify constraints on recommended attributes.

9.3. Constraints

A Constraint is a documented rule subject to validation that requires at least one of the specified recommended attributes of a class to be populated. Constraints are used in classes where there are attributes that cannot be required in all use cases, but in order to have unambiguous meaning, at least one of the attributes in the constraint is required. Attributes in a constraint must be Recommended.

The two constraints are:
– at_least_one
– just_one

These will be explained further in the section on Event Classes.

9.4. Attribute Groups

Attributes are grouped for documentation purposes into:
– Primary
– Classification
– Occurrence
– Context groups

Classification and Occurrence groupings are independent of event class and are defined with the attribute in the dictionary. Primary and Context attributes' groupings are based on their usage within a given event class.

Each event class has primary attributes, the attributes that are indicative of the event semantics in all use cases. Primary attributes are typically Required, or Recommended per event class, based on their use in each class. Primary attributes in the Base Event class apply to all event classes.

Attributes that are important for the taxonomy of the framework are designated as Classification attributes. The classification attributes are marked as Required as part of the Base Event class. Their values are nominally Unknown or Other and will be overridden within specific event classes.

Attributes that are related to time and time ranges are designated as Occurrence attributes. The occurrence attributes may be marked with any requirement level, depending on their usage within an event class.

Attributes that are used for variations on typical use cases, to enhance the meaning or enrich the content of an event are designated as Context attributes. The context attributes may be marked with any requirement level, but most often are marked as Optional.

9.5. Timestamp and Datetime Attributes

Representing time values is one of the most important aspects of the schema. For an event schema it is even more important. There are time attributes associated with events that need to be captured in a number of places throughout the schema, for example when a file was opened or when a process started and stopped. There are also times that are directly related to the event stream, for example event creation, collection, processing, and logging. The nominal data type for these attributes is timestamp_t based on Unix time or number of milliseconds since the Unix epoch. The datetime_t data type represents times in human readable RFC3339 form.

The Date/Time profile when applied adds a sibling attribute of data type datetime_t wherever a timestamp_t attribute appears in the schema.

The following terms are used below:
- Event Producer — the system (application, services, etc.) that generates events. Related to the producer persona.
- Event Consumer — the system that receives the events generated by the event producer. Related to the analyst persona.
- Event Processor — a system that processes and logs, including an ETL chain, the events received by the event consumer. Related to the mapper and analyst personas.

The core time attributes may be present in all events as they are from the Base Event class. They are:
- original_time: string The original event time, as created by the event producer as part of the Metadata object of the Base Event class. The time format is not specified by the schema and as such is a non-validated string. The time could be UTC time in milliseconds (1659378222123), ISO 8601 (2019-09-07T15:50-04:00), or any other value (12/13/2021 10:12:55 PM).
- time: timestamp_t The normalized event occurrence time. Normalized time means the original event time original_time is corrected for the clock skew of the source if any, and batch submission delay and after it was converted to the schema timestamp_t.
- processed_time: timestamp_t The time when the event (or batch of events) was sent by the event processor to the event consumer. The processed time can be used to determine the clock skew at the earliest known event source. Clock skew occurs when the UTC clock time on one computer differs from the UTC clock time on another computer. It is assumed that the transport latency is very small compared to the clock skew, therefore if the processed_time is very close to the logged_time, no correction should be made, notwithstanding any known hops.
- logged_time: timestamp_t The time when the event consumer logged the event. It must be equal or greater than the normalized event occurrence time. modified_time: timestamp_t The time when the event was last updated or enriched. It must be equal or greater than the normalized event occurrence time. It could be less-than, equal, or greater-than the logged_time .
- start_time/end_time: timestamp_t The start and end event times of the Base Event class are used when the event represents some activity that happened over a time range, for example a vulnerability or virus scan, or a discovery run. The other use-case is event aggregation. Aggregation is a mechanism that allows for a number of events of the same event type to be summarized into one for more efficient processing. For example netflow events. In this use case, the count integer attribute is also populated.

9.5.1. Time Zone

The time zone where the event occurred is represented by the timezone_offset attribute of data type Integer. Although time attributes are otherwise UTC except for the pass through attribute original_time, most security use cases benefit from knowing what time of day the event occurred at the event source.

timezone_offset is the number of minutes that the reported event time is ahead or behind UTC, in the range -1,080 to +1,080. It is a recommended attribute of the Base Event class.

9.6. Metadata

Metadata is an object referenced by the required Base Event attribute metadata. As its name implies, the attribute is populated with data outside of the source event. Some of the attributes of the object are optional, such as logged_time and uid, while the version attribute is required - the schema version for the event. It is expected that a logging system may assign the logged_time and uid at storage time.

Metadata attributes such as modified_time and processed_time are optional. modified_time is populated when an event has been enriched or mutated in some way before analysis or storage. processed_time is populated typically when an event is collected and submitted to a logging system.8

9.6.1. Version

The core schema version uses Semantic Versioning Specification (SemVer), e.g. 0.99.0, which indicates to consumers of the event which attributes may be found in the event, and what the class and category structure are. The convention is that the major version, after 1.0.0, or first part, remains the same while versions of the schema remain backwards compatible with previous versions of the schema and framework. As new classes, attributes, objects and profiles are added to the schema, the minor version, or second part of the version increases. The third part is reserved for corrections that don't break the schema, for example documentation or caption changes.

Extensions,discussed later, have their own versions and can change at their own pace but must remain compatible and consistent with the major version of the core schema that they extend. The optional extension attribute of type Schema Extension carries the version of an extension.

9.7. Observables

Observable is an object referenced by the primary Base Event class array attribute observables. It is populated from other attributes produced or mapped from the source event. An Observable object surfaces attribute information in one place irrespective of event class, while the security relevant indicators that populate the observable may occur in many places across event classes. In effect it is an array of summaries of those attributes regardless of where they stem from in the event based on their data type or object type (e.g. ip_address, process, file etc).

For example, an IP address may populate multiple attributes: public_ip, intermediate_ips, ip (as part of objects Endpoint, Device, Network Proxy, etc.). An analyst may be interested to know if a particular IP address is present anywhere in any event. Searching for the IP address value from the Base Event observables attribute surfaces any of these events more easily than remembering all of the attributes across all event classes that may have an IP address.

There are three important attributes in the Observable object: name, value, and type_id. For scalar attributes within an event, all three observable attributes are populated, where the type_id declares what the type of attribute is, the name is the fully qualified attribute name within the event, and value is the value of that attribute.

For complex (object type) attributes, Observable.name is the pointer or reference to the attribute, but as an object has more than one value, Observable.value is not populated.

```
"observables": [
  {
    "name": "actor.process.name",
    "type": "Process Name",
    "type_id": 9,
    "value": "Notepad.exe"
  },
  {
    "Name": "tls.ja3_hash",
    "Type": "Fingerprint",
    "Type_id": "30"
  },
  {
    "name": "file.name",
    "type": "File Name",
    "type_id": 7,
    "value": "Notepad.exe"
  }
]
```

Figure 1

9.8. Enrichments

Enrichment is an object referenced by the Base Event array attribute enrichments. An Enrichment object describes additional information added to the event during collection or event processing but before an immutable operation such as storage of the event. An example would be looking up location data on an IP address, or IOCs against a domain name or file hash.

Because enriching data can be extremely open-ended, the object uses generic string attributes along with a JSON data attribute that holds an arbitrary enrichment in a form known to the processing system. Similar to the Observable object, name and value attributes are required to point to the event class attribute that is being enriched. Unlike Observable, there is no predefined set of attributes that are tagged for enrichment, therefore only a recommended type attribute is specified (i.e. there is no type_id Enum).

Also unlike Observable, which is synchronized with the time of the event, it is assumed that there is some latency between the event time and the time the event is enriched, hence the Base Event class metadata.modified_time should be populated at the time of enrichment.

For example

```
"metadata": {
  "logged_time": 1659056959885810,
  "modified_time": 1659056959885807,
  "processed_time": 1659056959885796,
  "sequence": 69,
  "uid": "1310fc5c-0edb-11ed-88fc-0242ac110002",
  "version": "1.0.0-RC3"
},
"enrichments": [
  {
    "data": {
      "hash": 0c5ad1e8fe43583e279201cdb1046aea742bae59685e6da24e963a41df987494
    },
    "name": "ip",
    "provider": "provider.example.org",
    "type": "IP Address",
    "value": "103.216.221.19"
  },
```

```
  {
    "data": {
      "yara_rule": "wellmail_unique_strings \{ meta: description =
        "Rule for detection of WellMail based on unique strings contained in the
 binary"
        author = "XXXX" hash =
"0c5ad1e8fe43583e279201cdb1046aea742bae59685e6da24e963a41df987494"
        strings: $a = "C:\\Server\\Mail\\App_Data\\Temp\\agent.sh\\src"
        $b = "C:/Server/Mail/App_Data/Temp/agent.sh/src/main.go" $c =
"HgQdbx4qRNv"
        $d = "042a51567eea19d5aca71050b4535d33d2ed43ba" $e = "main.zipit"
        $f = "@[^\\s]+?\\s(?P.*?)\\s" condition: uint32(0) == 0x464C457F and 3
of them \}"
    },
    "name": "ip",
    "provider": "provider.example.org",
    "type": "IP Address",
    "value": "103.216.221.19"
  }
]
```

Figure 2

9.9. Dictionary of Attributes

The Attribute Dictionary defines attributes and includes references to the events and objects in which they are used.

The following list reports the Attribute Name and Attribute Description, along with their enumerations, in the form `<"Attribute Name">:<"Attribute Description">`, with sub-indexed enumerative items and corresponding descriptions where applicable

9.9.1. Access List

The list of requested access rights.

9.9.2. Access Mask

The access mask in a platform-native format.

9.9.3. Access Check Result

The list of access check results.

9.9.4. Accessed Time

The time when the file was last accessed.

9.9.5. Accessor

The name of the user who last accessed the object.

9.9.6. Account

The account object describes details about the account that was the source or target of the activity.

9.9.7. Acknowledgement Reason

An integer that provides a reason code or additional information about the acknowledgment result.

9.9.8. Acknowledgement Result

An integer that denotes the acknowledgment result of the DCE/RPC call.

9.9.9. Activity ID

The normalized identifier of the activity that triggered the event.

Table 1 — Activity ID enumerable values

Value	Name	Description
99	Other	The event activity is not mapped.
0	Unknown	The event activity is unknown.

9.9.10. Activity

The event activity name, as defined by the activity_id.

9.9.11. Actor

The actor object describes details about the user/role/process that was the source of the activity.

9.9.12. Actual Permissions

The permissions that were granted to the in a platform-native format.

9.9.13. Affected Code

List of Affected Code objects that describe details about code blocks identified as vulnerable.

9.9.14. Affected Software Packages

List of software packages identified as affected by a vulnerability/vulnerabilities.

9.9.15. Client TLS Alert

The integer value of TLS alert if present. The alerts are defined in the TLS specification in RFC-2246.

9.9.16. Algorithm

The applicable algorithm, normalized to the caption of 'algorithm_id'. See specific usage.

9.9.17. Algorithm ID

The identifier of the normalized algorithm. See specific usage.

Table 2 — Algorithm ID enumerable values

Value	Name	Description
99	Other	
0	Unknown	

9.9.18. Analytic

The analytic technique used to analyze and derive insights from the data or information that led to the finding or conclusion.

9.9.19. DNS Answer

The Domain Name System (DNS) answers.

9.9.20. API Details

Describes details about a typical API (Application Programming Interface) call.

9.9.21. Application

The application that reported the event.

9.9.22. Application Name

The name of the application that is associated with the event or object.

9.9.23. Architecture

Architecture is a shorthand name describing the type of computer hardware the packaged software is meant to run on.

9.9.24. HTTP Arguments

The arguments sent along with the HTTP request.

9.9.25. Attempt

The delivery attempt.

9.9.26. SMTP Banner

The initial SMTP connection response that a messaging server receives after it connects to a email server.

9.9.27. Attacks

An array of attacks associated with an event.

9.9.28. Attributes

The bitmask value that represents the file attributes.

9.9.29. Auth Protocol

The authentication protocol as defined by the caption of 'auth_protocol_id'. In the case of 'Other', it is defined by the event source.

9.9.30. Auth Protocol ID

The normalized identifier of the authentication protocol used to create the user session.

Table 3 — Auth Protocol ID enumerable values

Value	Name	Description
99	Other	
0	Unknown	
1	NTLM	
10	RADIUS	
2	Kerberos	
3	Digest	
4	OpenID	
5	SAML	
6	OAUTH 2.0	
7	PAP	
8	CHAP	
9	EAP	

9.9.31. Authentication Type

The agreed upon authentication type, normalized to the caption of 'auth_type_id'. In the case of 'Other', it is defined by the event source.

9.9.32. Authentication Type ID

The normalized identifier of the agreed upon authentication type. See specific usage.

Table 4 — Authentication Type ID enumerable values

Value	Name	Description
99	Other	
0	Unknown	

9.9.33. Authorization Information

This object provides details such as authorization outcome, associated policies related to activity/ event.

9.9.34. Autoscale UID

The unique identifier of the cloud autoscale configuration.

9.9.35. Base Address

The memory address where the module was loaded.

9.9.36. Base Score

The base score as reported by the event source. See specific usage.

9.9.37. BIOS Date

The BIOS date. For example: 03/31/16.

9.9.38. BIOS Manufacturer

The BIOS manufacturer. For example: LENOVO.

9.9.39. BIOS Version

The BIOS version. For example: LENOVO G5ETA2WW (2.62).

9.9.40. Boundary

The boundary of the connection, normalized to the caption of 'boundary_id'. In the case of 'Other', it is defined by the event source. For cloud connections, this translates to the traffic-boundary(same VPC, through IGW, etc.). For traditional networks, this is described as Local, Internal, or External.

9.9.41. Boundary ID

The normalized identifier of the boundary of the connection. For cloud connections, this translates to the traffic-boundary (same VPC, through IGW, etc.). For traditional networks, this is described as Local, Internal, or External.

Table 5 — Boundary ID enumerable values

Value	Name	Description
99	Other	
0	Unknown	The connection boundary is unknown.

Value	Name	Description
1	Localhost	Local network traffic on the same endpoint.
10	Gateway VPC	Through a gateway VPC endpoint (Nitro-based instances only)
11	Internet Gateway	Through an Internet gateway (Nitro-based instances only)
2	Internal	Internal network traffic between two endpoints inside network.
3	External	External network traffic between two endpoints on the Internet or outside the network.
4	Same VPC	Through another resource in the same VPC
5	Internet/VPC Gateway	Through an Internet gateway or a gateway VPC endpoint
6	Virtual Private Gateway	Through a virtual private gateway
7	Intra-region VPC	Through an intra-region VPC peering connection
8	Inter-region VPC	Through an inter-region VPC peering connection
9	Local Gateway	Through a local gateway

9.9.42. OS Build

The operating system build number.

9.9.43. Patch Bulletin

The vendor bulletin identfier.

9.9.44. Total Bytes

The total number of bytes (in and out).

9.9.45. Bytes In

The number of bytes sent from the destination to the source.

9.9.46. Bytes Out

The number of bytes sent from the source to the destination.

9.9.47. Capabilities

A list of RDP capabilities.

9.9.48. Caption

A short description or caption of the device. For example: Scanner 1 or Database Manager.

9.9.49. Website Categorization

The Website categorization names, as defined by category_ids enum values.

9.9.50. Category

The object category. See specific usage.

9.9.51. Website Categorization IDs

The Website categorization identifies.

9.9.52. Category

The event category name, as defined by category_uid value.

9.9.53. Category ID

The category unique identifier of the event.

9.9.54. Cc

The email header Cc values, as defined by RFC 5322.

9.9.55. Certificate

The certificate object containing information about the digital certificate.

9.9.56. Certificate Chain

The Chain of Certificate Serial Numbers field provides a chain of Certificate Issuer Serial Numbers leading to the Root Certificate Issuer.

9.9.57. Chassis

The chassis type describes the system enclosure or physical form factor. Such as the following examples for Windows Windows Chassis Types

9.9.58. Cipher Suite

The negotiated cipher suite.

9.9.59. CIS Benchmark Result

The CIS benchmark result.

9.9.60. CIS Benchmark

The CIS Benchmark describes best practices for securely configuring IT systems, software, networks, and cloud infrastructure as defined by the Center for Internet Security (CIS).

9.9.61. CIS CSC

The CIS Critical Security Controls is a list of top 20 actions and practices an organization's security team can take on such that cyber attacks or malware, are minimized and prevented.

9.9.62. CIS Controls

The CIS Critical Security Controls is a prioritized set of actions to protect your organization and data from cyber-attack vectors.

9.9.63. City

The name of the city.

9.9.64. Class

The class name of the object. See specific usage.

9.9.65. Class

The event class name, as defined by class_uid value.

9.9.66. Class ID

The unique identifier of a class. A Class describes the attributes available in an event.

9.9.67. Classification IDs

The list of normalized identifiers of the malware classifications. Reference: STIX Malware Types

9.9.68. Classification

The classification as defined by the vendor.

9.9.69. Classifications

The list of malware classifications, normalized to the captions of the classification_id values. In the case of 'Other', they are defined by the event source.

9.9.70. Client Cipher Suites

The client cipher suites that were exchanged during the TLS handshake negotiation.

9.9.71. Client Dialects

The list of SMB dialects that the client speaks.

9.9.72. Client HASSH

The Client HASSH fingerprinting object.

9.9.73. Cloud

Describes details about the Cloud environment where the event was originally created or logged.

9.9.74. Cloud Partition

The canonical cloud partition name to which the region is assigned (e.g. AWS Partitions: aws, aws-cn, aws-us-gov).

9.9.75. Command Line

The full command line used to launch an application, service, process, or job. For example: ssh user@10.0.0.10. If the command line is unavailable or missing, the empty string '' is to be used

9.9.76. Response Code

The numeric response sent to a request.

9.9.77. Response Codes

The list of numeric responses sent to a request.

9.9.78. Color Depth

The numeric color depth.

9.9.79. Command

The command name.

9.9.80. Command Response

The response to the command.

9.9.81. Command Responses

The responses to the command.

9.9.82. Comment

The user-provided comment.

9.9.83. Company Name

The name of the company that published the file. For example: Microsoft Corporation.

9.9.84. Compliance

The compliance object provides context to compliance findings (e.g., a check against a specific regulatory or best practice framework such as CIS, NIST etc.) and contains compliance related details.

9.9.85. Component

The name or relative pathname of a sub-component of the data object, if applicable. For example: attachment.doc, attachment.zip/bad.doc, or part.mime/part.cab/part.uue/part.doc.

9.9.86. Confidence

The confidence, normalized to the caption of the confidence_id value. In the case of 'Other', it is defined by the event source.

9.9.87. Confidence Id

The normalized confidence refers to the accuracy of the rule that created the finding. A rule with a low confidence means that the finding scope is wide and may create finding reports that may not be malicious in nature.

Table 6 — Confidence Id enumerable values

Value	Name	Description
0	Unknown	No confidence is assigned.
1	Low	
2	Medium	
3	High	
99	Other	The confidence is not mapped to the defined enum values. See the confidence attribute, which contains a data source specific value.

9.9.88. Confidence Score

The confidence score as reported by the event source.

9.9.89. Confidentiality

The file content confidentiality, normalized to the confidentiality_id value. In the case of 'Other', it is defined by the event source.

9.9.90. Confidentiality ID

The normalized identifier of the file content confidentiality indicator.

Table 7 — Confidentiality ID enumerable values

Value	Name	Description
99	Other	
0	Unknown	
1	Not Confidential	
2	Confidential	
3	Secret	
4	Top Secret	

9.9.91. Connection Info

The network connection information.

9.9.92. Connection Identifier

The network connection identifier.

9.9.93. Container

The information describing an instance of a container. A container is a prepackaged, portable system image that runs isolated on an existing system using a container runtime like containerd.

9.9.94. Containers

When working with containerized applications, the set of containers which write to the standard the output of a particular logging driver. For example, this may be the set of containers involved in handling api requests and responses for a containerized application.

9.9.95. HTTP Content Type

The request header that identifies the original media type of the resource (prior to any content encoding applied for sending).

9.9.96. Continent

The name of the continent.

9.9.97. Security Control

A Control is prescriptive, prioritized, and simplified set of best practices that one can use to strengthen their cybersecurity posture. e.g. AWS SecurityHub Controls, CIS Controls.

9.9.98. Coordinates

A two-element array, containing a longitude/latitude pair. The format conforms with GeoJSON. For example: [-73.983, 40.719].

9.9.99. Correlation UID

The unique identifier used to correlate events.

9.9.100. Cost Center

The cost center associated with the user.

9.9.101. Count

The number of times that events in the same logical group occurred during the event Start Time to End Time period.

9.9.102. Country

The ISO 3166-1 Alpha-2 country code. For the complete list of country codes see ISO 3166-1 alpha-2 codes.Note: The two letter country code should be capitalized. For example: US or CA.

9.9.103. The product CPE identifier

The Common Platform Enumeration (CPE) name as described by (NIST) For example: cpe:/a:apple:safari:16.2.

9.9.104. CPU Bits

The cpu architecture, the number of bits used for addressing in memory. For example: 32 or 64.

9.9.105. CPU Cores

The number of processor cores in all installed processors. For Example: 42.

9.9.106. CPU Count

The number of physical processors on a system. For example: 1.

9.9.107. Processor Speed

The speed of the processor in Mhz. For Example: 4200.

9.9.108. Processor Type

The processor type. For example: x86 Family 6 Model 37 Stepping 5.

9.9.109. Create Mask

The original Windows mask that is required to create the object.

9.9.110. Created Time

The time when the object was created. See specific usage.

9.9.111. Creator

The user that created the object associated with event. See specific usage.

9.9.112. User Credential ID

The unique identifier of the user's credential. For example, AWS Access Key ID.

9.9.113. Criticality

Criticality of a resource/object in question

9.9.114. Customer UID

The unique customer identifier.

9.9.115. CVE

The Common Vulnerabilities and Exposures (CVE).

9.9.116. CVE List

List of Common Vulnerabilities and Exposures (CVE).

9.9.117. CVSS Score

The CVSS object details Common Vulnerability Scoring System (CVSS) scores from the advisory that are related to the vulnerability.

9.9.118. CWE

The CWE object represents a weakness in a software system that can be exploited by a threat actor to perform an attack. The CWE object is based on the Common Weakness Enumeration (CWE) catalog.

9.9.119. Data

The additional data that is associated with the event or object. See specific usage.

9.9.120. Data Sources

The data sources for the finding.

9.9.121. Distributed Computing Environment/Remote Procedure Call (DCE/RPC)

The DCE/RPC object describes the remote procedure call system for distributed computing environments.

9.9.122. Authorization Decision/Outcome

Decision/outcome of the authorization mechanism (e.g. Approved, Denied)

9.9.123. Root Delay

The total round-trip delay to the reference clock in milliseconds.

9.9.124. Deleted Time

The timestamp when the user was deleted. In Active Directory (AD), when a user is deleted they are moved to a temporary container and then removed after 30 days. So, this field can be populated even after a user is deleted for the next 30 days.

9.9.125. Delivered To

The Delivered-To email header field.

9.9.126. CVSS Depth

The CVSS depth represents a depth of the equation used to calculate CVSS score.

Table 8 — CVSS Depth enumerable values

Value	Name	Description
`Base`	Base	
`Environmental`	Environmental	
`Temporal`	Temporal	

9.9.127. Description

The description that pertains to the object or event. See specific usage.

9.9.128. Desktop Display

The desktop display affiliated with the event

9.9.129. Details

Details of an entity. See specific usage

9.9.130. Detection UID

The associated unique detection event identifier. For example: detection response events include the Detection ID of the original event.

9.9.131. Developer UID

The developer ID on the certificate that signed the file.

9.9.132. Device

An addressable device, computer system or host.

9.9.133. Devices

The object describes details related to the list of devices.

9.9.134. Dialect

The negotiated protocol dialect.

9.9.135. Message Digest

The message digest attribute contains the fixed length message hash representation and the corresponding hashing algorithm information.

9.9.136. Direction

The direction of the initiated connection, traffic, or email, normalized to the caption of the direction_id value. In the case of 'Other', it is defined by the event source.

9.9.137. Direction ID

The normalized identifier of the direction of the initiated connection, traffic, or email.

Table 9 — Direction ID enumerable values

Value	Name	Description
0	Unknown	Connection direction is unknown.
1	Inbound	Inbound network connection. The connection was originated from the Internet or outside network, destined for services on the inside network.
2	Outbound	Outbound network connection. The connection was originated from inside the network, destined for services on the Internet or outside network.
3	Lateral	Lateral network connection. The connection was originated from inside the network, destined for services on the inside network.
99	Other	

9.9.138. Root Dispersion

The dispersion in the NTP protocol is the estimated time error or uncertainty relative to the reference clock in milliseconds.

9.9.139. Disposition

The event disposition name, normalized to the caption of the disposition_id value. In the case of 'Other', it is defined by the event source.

9.9.140. Disposition ID

When security issues, such as malware or policy violations, are detected and possibly corrected, then disposition_id describes the action taken by the security product.

Table 10 — Disposition ID enumerable values

Value	Name	Description
99	Other	
0	Unknown	

9.9.141. DKIM Status

The DomainKeys Identified Mail (DKIM) status of the email.

9.9.142. DKIM Domain

The DomainKeys Identified Mail (DKIM) signing domain of the email.

9.9.143. DKIM Signature

The DomainKeys Identified Mail (DKIM) signature used by the sending/receiving system.

9.9.144. DMARC Status

The Domain-based Message Authentication, Reporting and Conformance (DMARC) status of the email.

9.9.145. DMARC Override

The Domain-based Message Authentication, Reporting and Conformance (DMARC) override action.

9.9.146. DMARC Policy

The Domain-based Message Authentication, Reporting and Conformance (DMARC) policy status.

9.9.147. Domain

The name of the domain.

9.9.148. Kernel Driver

The driver that was loaded/unloaded into the kernel

9.9.149. Destination Endpoint

The network destination endpoint.

9.9.150. Duration

The event duration or aggregate time, the amount of time the event covers from start_time to end_time in milliseconds.

9.9.151. OS Edition

The operating system edition. For example: Professional.

9.9.152. Email

The email object.

9.9.153. Email Address

The user's primary email address.

9.9.154. Email Addresses

A list of additional email addresses for the user.

9.9.155. Email Authentication

The SPF, DKIM and DMARC attributes of an email.

9.9.156. Email UID

The unique identifier of the email, used to correlate related email alert and activity events.

9.9.157. Employee ID

The employee identifier assigned to the user by the organization.

9.9.158. End Line

The line number of the last line of code block identified as vulnerable.

9.9.159. End Time

The end time of a time period. See specific usage.

9.9.160. Enrichments

The additional information from an external data source, which is associated with the event. For example add location information for the IP address in the DNS answers:[{"name": "answers.ip", "value": "92.24.47.250", "type": "location", "data": {"city": "Socotra", "continent": "Asia", "coordinates": [-25.4153, 17.0743], "country": "YE", "desc": "Yemen"}}]

9.9.161. Entity

The managed entity that is being acted upon.

9.9.162. Entity Result

The updated managed entity.

9.9.163. Epoch

The software package epoch. Epoch is a way to define weighted dependencies based on version numbers.

9.9.164. EPSS

The Exploit Prediction Scoring System (EPSS) object describes the estimated probability a vulnerability will be exploited. EPSS is a community-driven effort to combine descriptive information about vulnerabilities (CVEs) with evidence of actual exploitation in-the-wild. (EPSS).

9.9.165. Error Code

Error Code

9.9.166. Error Message

Error Message

9.9.167. Event Code

The Event ID or Code that the product uses to describe the event.

9.9.168. Evidence

The data the finding exposes to the analyst.

9.9.169. Exit Code

The exit code reported by a process when it terminates. The convention is that zero indicates success and any non-zero exit code indicates that some error occurred.

9.9.170. Expiration Time

The expiration time. See specific usage.

9.9.171. Schema Extension

The schema extension used to create the event.

9.9.172. Schema Extensions

The schema extensions used to create the event.

9.9.173. Extension List

The list of TLS extensions.

9.9.174. Feature

The feature that reported the event.

9.9.175. File

The file that pertains to the event or object. See specific usage.

9.9.176. File Diff

File content differences used for change detection. For example, a common use case is to identify itemized changes within INI or configuration/property setting values.

9.9.177. File Result

The result of the file change. It should contain the new values of the changed attributes.

9.9.178. Finding

The Finding object provides details about a finding/detection generated by a security tool.

9.9.179. Finding Information

The information about a finding or detection generated by a security tool.

9.9.180. Fingerprint

The digital fingerprint associated with an object.

9.9.181. Fingerprints

An array of digital fingerprint objects.

9.9.182. First Seen

The initial detection time of the activity or object. See specific usage

9.9.183. Fixed In Version

The software package version in which a reported vulnerability was patched/fixed.

9.9.184. Fix Availability

Indicates if a fix is available for the reported vulnerability.

9.9.185. Communication Flag IDs

The list of normalized identifiers of the communication flag IDs.

9.9.186. Flags

The list of communication flags, normalized to the captions of the flag_ids values. In the case of 'Other', they are defined by the event source.

9.9.187. Folder

The folder that pertains to the event.

9.9.188. From

The email header From values, as defined by RFC 5322.

9.9.189. Full Name

The full name of the person, as per the LDAP Common Name attribute (cn).

9.9.190. Function Keys

The number of function keys on client keyboard.

9.9.191. Function Name

The entry-point function of the module. The system calls the entry-point function whenever a process or thread loads or unloads the module.

9.9.192. Given Name

The given or first name of the user.

9.9.193. Group

The group object associated with an entity such as user, policy, or rule.

9.9.194. Group Name

The name of the group that the resource belongs to.

9.9.195. Groups

The groups to which an entity belongs. See specific usage.

9.9.196. Handshake Duration

The amount of total time for the TLS handshake to complete after the TCP connection is established, including client-side delays, in milliseconds.

9.9.197. Hash

The hash attribute is the value of a digital fingerprint including information about its algorithm.

9.9.198. Hashes

An array of hash attributes.

9.9.199. Hire Time

The timestamp when the user was or will be hired by the organization.

9.9.200. Hostname

The hostname of an endpoint or a device.

9.9.201. HTTP Cookies

The cookies object describes details about HTTP cookies

9.9.202. HTTP Headers

Additional HTTP headers of an HTTP request or response.

9.9.203. HTTP Method

The HTTP request method indicates the desired action to be performed for a given resource. Expected values: TRACE CONNECT OPTIONS HEAD DELETE POST PUT GET

9.9.204. HTTP Only

A cookie attribute to make it inaccessible via JavaScript

9.9.205. HTTP Request

The HTTP Request Object documents attributes of a request made to a web server.

9.9.206. HTTP Response

The HTTP Response from a web server to a requester.

9.9.207. HTTP Status

The Hypertext Transfer Protocol (HTTP) status code returned to the client.

9.9.208. Hardware Info

The device hardware information.

9.9.209. Hypervisor

The name of the hypervisor running on the device. For example, Xen, VMware, Hyper-V, VirtualBox, etc.

9.9.210. Identifier Cookie

The client identifier cookie during client/server exchange.

9.9.211. Identity Provider

This object describes details about the Identity Provider used.

9.9.212. Image

The image used as a template to run a container or virtual machine.

9.9.213. IME

The Input Method Editor (IME) file name.

9.9.214. IMEI

The International Mobile Station Equipment Identifier that is associated with the device.

9.9.215. Impact

The impact , normalized to the caption of the impact_id value. In the case of 'Other', it is defined by the event source.

9.9.216. Impact ID

The normalized impact of the finding.

Table 11 — Impact ID enumerable values

Value	Name	Description
0	Unknown	
1	Low	
2	Medium	
3	High	
4	Critical	
99	Other	The detection impact is not mapped. See the impact attribute, which contains a data source specific value.

9.9.217. Impact

The impact of the finding, valid range 0-100.

9.9.218. Injection Type

The process injection method, normalized to the caption of the injection_type_id value. In the case of 'Other', it is defined by the event source.

9.9.219. Injection Type ID

The normalized identifier of the process injection method.

Table 12 — Injection Type ID enumerable values

Value	Name	Description
99	Other	
0	Unknown	
1	Remote Thread	
2	Load Library	

9.9.220. Instance ID

The unique identifier of a VM instance.

9.9.221. Integrity

The process integrity level, normalized to the caption of the direction_id value. In the case of 'Other', it is defined by the event source (Windows only).

9.9.222. Integrity Level

The normalized identifier of the process integrity level (Windows only).

9.9.223. Network Interface Name

The name of the network interface (e.g. eth2).

9.9.224. Network Interface ID

The unique identifier of the network interface.

9.9.225. Intermediate IP Addresses

The intermediate IP Addresses. For example, the IP addresses in the HTTP X-Forwarded-For header.

9.9.226. Invoked by

The name of the service that invoked the activity as described in the event.

9.9.227. IP Address

The IP address, in either IPv4 or IPv6 format.

9.9.228. Cleartext Credentials

Indicates whether the credentials were passed in clear text.Note: True if the credentials were passed in a clear text protocol such as FTP or TELNET, or if Windows detected that a user's logon password was passed to the authentication package in clear text.

9.9.229. Compliant Device

The event occurred on a compliant device.

9.9.230. Default Value

The indication of whether the value is from a default value name. For example, the value name could be missing.

9.9.231. Exploit Availability

Indicates if an exploit or a PoC (proof-of-concept) is available for the reported vulnerability.

9.9.232. Fix Availability

Indicates if a fix is available for the reported vulnerability.

9.9.233. Managed Device

The event occurred on a managed device.

9.9.234. Multi Factor Authentication

Indicates whether Multi Factor Authentication was used during authentication.

9.9.235. New Logon

Indicates logon is from a device not seen before or a first time account logon.

9.9.236. On Premises

The indication of whether the location is on premises.

9.9.237. Personal Device

The event occurred on a personal device.

9.9.238. Remote

The indication of whether the session is remote.

9.9.239. Renewal

The indication of whether this is a lease/session renewal event.

9.9.240. System

The indication of whether the object is part of the operating system.

9.9.241. Trusted Device

The event occurred on a trusted device.

9.9.242. ISP

The name of the Internet Service Provider (ISP).

9.9.243. Issuer Details

The identifier of the issuer. See specific usage.

9.9.244. JA3 Hash

The MD5 hash of a JA3 string.

9.9.245. JA3S Hash

The MD5 hash of a JA3S string.

9.9.246. Job

The job object that pertains to the event.

9.9.247. Job Title

The user's job title.

9.9.248. Knowledgebase Articles

The KB article/s related to the entity. A KB Article contains metadata that describes the patch or an update.

9.9.249. Kernel

The kernel resource object that pertains to the event.

9.9.250. Key Length

The length of the encryption key.

9.9.251. Keyboard Information

The keyboard detailed information.

9.9.252. Keyboard Layout

The keyboard locale identifier name (e.g., en-US).

9.9.253. Keyboard Subtype

The keyboard numeric code.

9.9.254. Keyboard Type

The keyboard type (e.g., xt, ico).

9.9.255. Kill Chain

The Cyber Kill Chain® provides a detailed description of each phase and its associated activities within the broader context of a cyber attack.

9.9.256. Labels

The list of labels attached to an event, object, or attribute.

9.9.257. Language

The two letter lower case language codes, as defined by ISO 639-1. For example: en (English), de (German), or fr (French).

9.9.258. Last Login

The last time when the user logged in.

9.9.259. Last Run

The last run time of application or service. See specific usage.

9.9.260. Last Seen

The most recent detection time of the activity or object. See specific usage.

9.9.261. Latency

The HTTP response latency measured in milliseconds.

9.9.262. LDAP Common Name

The LDAP and X.500 commonName attribute, typically the full name of the person. For example, John Doe.

9.9.263. LDAP Distinguished Name

The X.500 Distinguished Name (DN) is a structured string that uniquely identifies an entry, such as a user, in an X.500 directory service For example, cn=John Doe,ou=People,dc=example,dc=com.

9.9.264. Lease Duration

This represents the length of the DHCP lease in seconds. This is present in DHCP Ack events.

9.9.265. Leave Time

The timestamp when the user left or will be leaving the organization.

9.9.266. Response Length

The HTTP response length, in number of bytes.

9.9.267. Lineage

The lineage of the process, represented by a list of paths for each ancestor process. For example: ['/usr/sbin/sshd', '/usr/bin/bash', '/usr/bin/whoami'].

9.9.268. Software License

The name or identifier of the license applied on package or software. See SPDX License List.

9.9.269. Load Type

The load type, normalized to the caption of the load_type_id value. In the case of 'Other', it is defined by the event source. It describes how the module was loaded in memory.

9.9.270. Load Type ID

The normalized identifier of the load type. It identifies how the module was loaded in memory.

9.9.271. Loaded Modules

The list of loaded module names.

9.9.272. Geo Location

The detailed geographical location usually associated with an IP address.

9.9.273. Log Level

The audit level at which an event was generated.

9.9.274. Log Name

The event log name. For example, syslog file name or Windows logging subsystem: Security.

9.9.275. Log Provider

The logging provider or logging service that logged the event. For example, Microsoft-Windows-Security-Auditing.

9.9.276. Log Version

The event log schema version that specifies the format of the original event. For example syslog version or Cisco Log Schema Version.

9.9.277. Logged Time

The time when the logging system collected and logged the event.This attribute is distinct from the event time in that event time typically contain the time extracted from the original event. Most of the time, these two times will be different.

9.9.278. Loggers

An array of Logger objects that describe the devices and logging products between the event source and its eventual destination. Note, this attribute can be used when there is a complex end-to-end path of event flow.

9.9.279. Logon Process

The trusted process that validated the authentication credentials.

9.9.280. Logon Type

The logon type, normalized to the caption of the logon_type_id value. In the case of 'Other', it is defined by the event source.

9.9.281. Logon Type ID

The normalized logon type identifier.

Table 13 — Logon Type ID enumerable values

Value	Name	Description
99	Other	Other logon type.

Value	Name	Description
0	System	Used only by the System account, for example at system startup.
10	Remote Interactive	A remote logon using Terminal Services or remote desktop application.
11	Cached Interactive	A user logged on to this device with network credentials that were stored locally on the device and the domain controller was not contacted to verify the credentials.
12	Cached Remote Interactive	Same as Remote Interactive. This is used for internal auditing.
13	Cached Unlock	Workstation logon.
2	Interactive	A local logon to device console.
3	Network	A user or device logged onto this device from the network.
4	Batch	A batch server logon, where processes may be executing on behalf of a user without their direct intervention.
5	OS Service	A logon by a service or daemon that was started by the OS.
7	Unlock	A user unlocked the device.
8	Network Cleartext	A user logged on to this device from the network. The user's password in the authentication package was not hashed.
9	New Credentials	A caller cloned its current token and specified new credentials for outbound connections. The new logon session has the same local identity, but uses different credentials for other network connections.

9.9.282. MAC Address

The Media Access Control (MAC) address that is associated with the network interface.

9.9.283. Malware

The list of malware identified by a finding.

9.9.284. Manager

The user's manager. This helps in understanding an org hierarchy. This should only ever be populated once in an event. I.e. there should not be a manager's manager in an event.

9.9.285. Message

The description of the event, as defined by the event source.

9.9.286. Message UID

The email header Message-Id value, as defined by RFC 5322.

9.9.287. Metadata

The metadata associated with the event.

9.9.288. Metrics

The general purpose metrics associated with the event. See specific usage.

9.9.289. MIME type

The Multipurpose Internet Mail Extensions (MIME) type of the file, if applicable.

9.9.290. Modified Time

The time when the object was last modified. See specific usage.

9.9.291. Modifier

The user that last modified the object associated with the event. See specific usage.

9.9.292. Module

The module that pertains to the event.

9.9.293. Name

The name of the entity. See specific usage.

9.9.294. Namespace

The namespace is useful in merger or acquisition situations. For example, when similar entities exist that you need to keep separate.

9.9.295. Namespace PID

If running under a process namespace (such as in a container), the process identifier within that process namespace.

9.9.296. Network Driver

The network driver used by the container. For example, bridge, overlay, host, none, etc.

9.9.297. Network Interfaces

The network interfaces that are associated with the device, one for each unique MAC address/IP address/hostname/name combination.Note: The first element of the array is the network information that pertains to the event.

9.9.298. Next Run

The next run time. See specific usage.

9.9.299. NIST List

The NIST Cybersecurity Framework recommendations for managing the cybersecurity risk.

9.9.300. Observables

The observables associated with the event.

9.9.301. Office Location

The primary office location associated with the user. This could be any string and isn't a specific address. For example, South East Virtual.

9.9.302. DNS Opcode

The DNS opcode specifies the type of the query message.

9.9.303. DNS Opcode ID

The DNS opcode ID specifies the normalized query message type.

Table 14 — DNS Opcode ID enumerable values

Value	Name	Description
0	Query	Standard query
1	Inverse Query	Inverse query, obsolete
2	Status	Server status request
3	Reserved	Reserved, not used
4	Notify	Zone change notification
5	Update	Dynamic DNS update
6	DSO Message	DNS Stateful Operations (DSO)

9.9.304. Open Mask

The Windows options needed to open a registry key.

9.9.305. Open Type

The file open type.

9.9.306. Operation

Verb/Operation associated with the request

9.9.307. Opnum

An operation number used to identify a specific remote procedure call (RPC) method or a method in an interface.

9.9.308. Organization

Organization and org unit relevant to the event or object.

9.9.309. Orchestrator

The orchestrator managing the container, such as ECS, EKS, K8s, or OpenShift.

9.9.310. Org Unit Name

The name of the organizational unit, within an organization. For example, Finance, IT, R&D

9.9.311. Org Unit ID

The alternate identifier for an entity's unique identifier. For example, its Active Directory OU DN or AWS OU ID.

9.9.312. Original Time

The original event time as reported by the event source. For example, the time in the original format from system event log such as Syslog on Unix/Linux and the System event file on Windows. Omit if event is generated instead of collected via logs.

9.9.313. OS

The device operating system.

9.9.314. Overall Score

The overall score as reported by the event source. See specific usage.

9.9.315. Owner

The user that owns the file/object.

9.9.316. Software Package

The Software Package object describes details about a software package. Defined by D3FEND d3f:SoftwarePackage.

9.9.317. Software Packages

List of vulnerable packages as identified by the security product

9.9.318. Package Manager

The software packager manager utilized to manage a package on a system, e.g. npm, yum, dpkg etc.

9.9.319. Packet UID

The packet identifier assigned by the protocol.

9.9.320. Total Packets

The total number of packets (in and out).

9.9.321. Packets In

The number of packets sent from the destination to the source.

9.9.322. Packets Out

The number of packets sent from the source to the destination.

9.9.323. Parent Folder

The parent folder in which the file resides. For example: c:\windows\system32

9.9.324. Parent Process

The parent process of this process object. It is recommended to only populate this field for the first process object, to prevent deep nesting.

9.9.325. Path

The path that pertains to the event or object. See specific usage.

9.9.326. Permission

The IAM permission related to an event

9.9.327. Kill Chain Phase

The cyber kill chain phase.

9.9.328. Kill Chain Phase ID

The cyber kill chain phase identifier.

Table 15 — Kill Chain Phase ID enumerable values

Value	Name	Description
0	Unknown	The kill chain phase is unknown.
1	Reconnaissance	The attackers pick a target and perform a detailed analysis, start collecting information (email addresses, conferences information, etc.) and evaluate the victim's vulnerabilities to determine how to exploit them.
2	Weaponization	The attackers develop a malware weapon and aim to exploit the discovered vulnerabilities.
3	Delivery	The intruders will use various tactics, such as phishing, infected USB drives, etc.
4	Exploitation	The intruders start leveraging vulnerabilities to executed code on the victim's system.
5	Installation	The intruders install malware on the victim's system.
6	Command & Control	Malware opens a command channel to enable the intruders to remotely manipulate the victim's system.
7	Actions on Objectives	With hands-on keyboard access, intruders accomplish the mission's goal.
99	Other	

9.9.329. Phones

The phone numbers associated with the user

9.9.330. Physical Height

The numeric physical height of display.

9.9.331. Physical Orientation

The numeric physical orientation of display.

9.9.332. Physical Width

The numeric physical width of display.

9.9.333. Process ID

The process identifier, as reported by the operating system. Process ID (PID) is a number used by the operating system to uniquely identify an active process.

9.9.334. Pod UUID

The unique identifier of the pod (or equivalent) that the container is executing on.

9.9.335. Policy

Describes details of a policy. See specific usage.

9.9.336. Port

The TCP/UDP port number associated with a connection. See specific usage.

9.9.337. Postal Code

The postal code of the location.

9.9.338. Precision

The numeric precision. See specific usage.

9.9.339. Priority

The priority, normalized to the caption of the priority_id value. In the case of 'Other', it is defined by the event source. See specific usage.

9.9.340. Priority ID

The normalized priority. See specific usage.

9.9.341. Privileges

The user or group privileges.

9.9.342. Process

The process object.

9.9.343. Processed Time

The event processed time, such as an ETL operation.

9.9.344. Product

The product that reported the event.

9.9.345. Product Identifier

Unique Identifier of a product.

9.9.346. Profiles

The list of profiles used to create the event.

9.9.347. Project ID

The unique identifier of a Cloud project.

9.9.348. Protocol Name

The TCP/IP protocol name in lowercase, as defined by the Internet Assigned Numbers Authority (IANA). See Protocol Numbers. For example: tcp or udp.

9.9.349. Protocol Number

The TCP/IP protocol number, as defined by the Internet Assigned Numbers Authority (IANA). Use -1 if the protocol is not defined by IANA. See Protocol Numbers. For example: 6 for TCP and 17 for UDP.

9.9.350. Protocol Version

The Protocol version, normalized to the caption of the protocol_ver_id value. In the case of 'Other', it is defined by the event source.

9.9.351. Protocol Version ID

The normalized identifier of the Protocol version.

9.9.352. Provider

The origin of information associated with the event. See specific usage.

9.9.353. Proxy

The proxy (server) in a network connection.

9.9.354. Package URL

A purl is a URL string used to identify and locate a software package in a mostly universal and uniform way across programing languages, package managers, packaging conventions, tools, APIs and databases.

9.9.355. Proxy Connection Info

The connection information from the proxy server to the remote server.

9.9.356. Proxy HTTP Request

The HTTP Request from the proxy server to the remote server.

9.9.357. Proxy HTTP Response

The HTTP Response from the remote server to the proxy server.

9.9.358. Proxy TLS

The TLS protocol negotiated between the proxy server and the remote server.

9.9.359. Proxy Traffic

The network traffic refers to the amount of data moving across a network, from proxy to remote server at a given point of time.

9.9.360. DNS Query

The Domain Name System (DNS) query.

9.9.361. HTTP Query String

The query portion of the URL. For example: the query portion of the URL http://www.example.com/search?q=bad&sort=date is q=bad&sort=date.

9.9.362. Query Time

The Domain Name System (DNS) query time.

9.9.363. RAM Size

The total amount of installed RAM, in Megabytes. For example: 2048.

9.9.364. Raw Header

The email authentication header.

9.9.365. Raw Data

The event data as received from the event source.

9.9.366. Response Code

The server response code, normalized to the caption of the rcode_id value. In the case of 'Other', it is defined by the event source.

9.9.367. Response Code ID

The normalized identifier of the server response code. See specific usage.

Table 16 — Response Code ID enumerable values

Value	Name	Description
99	Other	The dns response code is not defined by the RFC.
0	Unknown	

9.9.368. DNS RData

The data describing the DNS resource. The meaning of this data depends on the type and class of the resource record.

9.9.369. References

A list of reference URLs supporting the finding/detection.

9.9.370. HTTP Referrer

The request header that identifies the address of the previous web page, which is linked to the current web page or resource being requested.

9.9.371. Region

The name or the code of a region. See specific usage.

9.9.372. Domain Registrar

The domain registrar.

9.9.373. Related Analytics

Describes analytics related to the analytic of a finding or detection as identified by the security product.

9.9.374. Related Events

Describes events related to a finding or detection as identified by the security product.

9.9.375. Related Vulnerabilities

List of vulnerabilities that are related to this vulnerability.

9.9.376. Relay

The network relay that is associated with the event.

9.9.377. Software Release Details

Release is the number of times a version of the software has been packaged.

9.9.378. Remediation

The Remediation object describes the recommended remediation steps to address identified issue(s).

9.9.379. Remote Display

The remote display affiliated with the event

9.9.380. Reply To

The email header Reply-To values, as defined by RFC 5322.

9.9.381. Reputation Scores

Contains the original and normalized reputation scores.

9.9.382. API Request Details

General Purpose API Request Object. See specific usage

9.9.383. Requested Permissions

The permissions mask that were requested by the process.

9.9.384. Compliance Requirements

A list of requirements associated to a specific control in an industry or regulatory framework. e.g. NIST.800-53.r5 AU-10

9.9.385. Resource

The target resource.

9.9.386. Resource Type

The resource type as defined by the event source.

9.9.387. Resources Array

Describes details about resources that were affected by the activity/event.

9.9.388. Resource Results Array

Updated resources after an activity/event.

9.9.389. API Response Details

General Purpose API Response Object. See specific usage.

9.9.390. Response Time

The Domain Name System (DNS) response time.

9.9.391. Risk Level

The risk level, normalized to the caption of the risk_level_id value. In the case of 'Other', it is defined by the event source.

9.9.392. Risk Level ID

The normalized risk level id.

Table 17 — Risk Level ID enumerable values

Value	Name	Description
0	Info	
1	Low	
2	Medium	
3	High	
4	Critical	

9.9.393. Risk Score

The risk score as reported by the event source.

9.9.394. Remote Procedure Call Interface

The RPC Interface object describes the details pertaining to the remote procedure call interface.

9.9.395. Rule

The rules that reported the events.

9.9.396. Firewall Rule

The firewall rules that triggered the events.

9.9.397. Run State

The state of the job or service, normalized to the caption of the run_state_id value. In the case of 'Other', it is defined by the event source. See specific usage.

9.9.398. Run State ID

The normalized identifier of the state of the job or service. See specific usage.

9.9.399. Runtime

The backend running the container, such as containerd or cri-o.

9.9.400. SameSite

The cookie attribute that lets servers specify whether/when cookies are sent with cross-site requests. Values are: Strict, Lax or None

9.9.401. Sandbox

The name of the containment jail (i.e., sandbox). For example, hardened_ps, high_security_ps, oracle_ps, netsvcs_ps, or default_ps.

9.9.402. Subject Alternative Names

The list of subject alternative names that are secured by a specific certificate.

9.9.403. Scale Factor

The numeric scale factor of display.

9.9.404. Scheme

The scheme portion of the URL. For example: http, https, ftp, or sftp.

9.9.405. Reputation Score

The reputation score, normalized to the caption of the score_id value. In the case of 'Other', it is defined by the event source.

9.9.406. Reputation Score ID

The normalized reputation score identifier.

Table 18 — Reputation Score ID enumerable values

Value	Name	Description
99	Other	The reputation score is not mapped. See the rep_score attribute, which contains a data source specific value.
0	Unknown	The reputation score is unknown.
1	Very Safe	Long history of good behavior.
10	Malicious	Proven evidence of maliciousness.
2	Safe	Consistently good behavior.
3	Probably Safe	Reasonable history of good behavior.
4	Leans Safe	Starting to establish a history of normal behavior.
5	May not be Safe	No established history of normal behavior.
6	Exercise Caution	Starting to establish a history of suspicious or risky behavior.
7	Suspicious/ Risky	A site with a history of suspicious or risky behavior. (spam, scam, potentially unwanted software, potentially malicious).
8	Possibly Malicious	Strong possibility of maliciousness.
9	Probably Malicious	Indicators of maliciousness.

9.9.407. Secure

The cookie attribute to only send cookies to the server with an encrypted request over the HTTPS protocol.

9.9.408. Security Descriptor

The object security descriptor.

9.9.409. Sequence Number

Sequence number of the event. The sequence number is a value available in some events, to make the exact ordering of events unambiguous, regardless of the event time precision.

9.9.410. Serial Number

The serial number that pertains to the object. See specific usage.

9.9.411. Server Cipher Suites

The server cipher suites that were exchanged during the TLS handshake negotiation.

9.9.412. Server HASSH

The Server HASSH fingerprinting object.

9.9.413. Service

The service that pertains to the event.

9.9.414. Session

The authenticated user or service session.

9.9.415. Severity

The event severity, normalized to the caption of the severity_id value. In the case of 'Other', it is defined by the event source.

9.9.416. Severity ID

The normalized identifier of the event severity.The normalized severity is a measurement the effort and expense required to manage and resolve an event or incident. Smaller numerical values represent lower impact events, and larger numerical values represent higher impact events.

Table 19 — Severity ID enumerable values

Value	Name	Description
99	Other	The event severity is not mapped. See the severity attribute, which contains a data source specific value.
0	Unknown	The event severity is not known.
1	Informational	Informational message. No action required.
2	Low	The user decides if action is needed.
3	Medium	Action is required but the situation is not serious at this time.
4	High	Action is required immediately.
5	Critical	Action is required immediately and the scope is broad.
6	Fatal	An error occurred but it is too late to take remedial action.

9.9.417. Share

The SMB share name.

9.9.418. Share Type

The SMB share type, normalized to the caption of the share_type_id value. In the case of 'Other', it is defined by the event source.

9.9.419. Share Type Id

The normalized identifier of the SMB share type.

Table 20 — Share Type Id enumerable values

Value	Name	Description
99	Other	The share type is not mapped.
0	Unknown	The share type is not known.
1	File	
2	Pipe	
3	Print	

9.9.420. Digital Signature

The digital signature of the file.

9.9.421. Size

The size of data, in bytes.

9.9.422. SMTP From

The value of the SMTP MAIL FROM command.

9.9.423. SMTP Hello

The value of the SMTP HELO or EHLO command.

9.9.424. SMTP To

The value of the SMTP envelope RCPT TO command.

9.9.425. Server Name Indication

`The Server Name Indication (SNI) extension sent by the client.`

Figure 3

9.9.426. SPF Status

The Sender Policy Framework (SPF) status of the email.

9.9.427. OS Service Pack

The name of the latest Service Pack.

9.9.428. OS Service Pack Version

The version number of the latest Service Pack.

9.9.429. Source Endpoint

The network source endpoint.

9.9.430. Source URL

The URL pointing towards the source of an entity. See specific usage.

9.9.431. Security Standards

Security standards are a set of criteria organizations can follow to protect sensitive and confidential information. e.g. NIST SP 800-53, CIS AWS Foundations Benchmark v1.4.0, ISO/IEC 27001

9.9.432. Start Address

The start address of the execution.

9.9.433. Start Line

The line number of the first line of code block identified as vulnerable.

9.9.434. Start Time

The start time of a time period. See specific usage.

9.9.435. State

The state of the event or object, normalized to the caption of the state_id value. In the case of 'Other', it is defined by the event source. See specific usage.

9.9.436. State ID

The normalized state ID of the event or object. See specific usage.

Table 21 — State ID enumerable values

Value	Name	Description
99	Other	The state is not mapped. See the state attribute, which contains a data source specific value.
0	Unknown	

9.9.437. Status

The event status, normalized to the caption of the status_id value. In the case of 'Other', it is defined by the event source.

9.9.438. Status Code

The event status code, as reported by the event source.For example, in a Windows Failed Authentication event, this would be the value of 'Failure Code', e.g. 0x18.

9.9.439. Status Details

The status details contains additional information about the event outcome.

9.9.440. Status ID

The normalized identifier of the event status.

Table 22 — Status ID enumerable values

Value	Name	Description
99	Other	The event status is not mapped. See the status attribute, which contains a data source specific value.
0	Unknown	
1	Success	
2	Failure	

9.9.441. Stratum

The stratum level of the NTP server's time source, normalized to the caption of the stratum_id value.

9.9.442. Stratum ID

The normalized identifier of the stratum level, as defined in RFC-5905.

Table 23 — Stratum ID enumerable values

Value	Name	Description
0	Unknown	Unspecified or invalid.
1	Primary Server	The highest precision primary server (e.g atomic clock or GPS).
2	Secondary Server	A secondary level server (possible values: 2-15).
16	Unsynchronized	
17	Reserved	Reserved stratum (possible values: 17-255).
99	Other	The stratum level is not mapped. See the stratum attribute, which may contain a data source specific value.

9.9.443. Subdomain

The subdomain portion of the URL. For example: sub in https://sub.example.com or sub2.sub1 in https://sub2.sub1.example.com.

9.9.444. Subject Details

The identifier of the subject. See specific usage.

9.9.445. Subnet

The subnet mask.

9.9.446. Subnet Prefix Length

The subnet prefix length determines the number of bits used to represent the network part of the IP address. The remaining bits are reserved for identifying individual hosts within that subnet.

9.9.447. Subnet UID

The unique identifier of a virtual subnet.

9.9.448. Supporting Data

Additional data supporting a finding as provided by security tool

9.9.449. The patch is superseded.

The vendor patch has been replaced by another.

9.9.450. Surname

The last or family name for the user.

9.9.451. Service Name

The service name in service-to-service connections. For example, AWS VPC logs the pkt-src-aws-service and pkt-dst-aws-service fields identify the connection is coming from or going to an AWS service.

9.9.452. System Call

The system call that was invoked.

9.9.453. Tactics

The a list of tactic ID's/names that are associated with the attack technique, as defined by ATT&CK MatrixTM.

9.9.454. Image Tag

The image tag. For example: 1.11-alpine.

9.9.455. TCP Flags

The network connection TCP header flags (i.e., control bits).

9.9.456. Technique

The attack technique.

9.9.457. Terminal

The Pseudo Terminal. Ex: the tty or pts value.

9.9.458. Terminated Time

The time when the entity was terminated. See specific usage.

9.9.459. Thread ID

The Identifier of the thread associated with the event, as returned by the operating system.

9.9.460. Event Time

The normalized event occurrence time.

9.9.461. Timezone Offset

The number of minutes that the reported event time is ahead or behind UTC, in the range -1,080 to +1,080.

9.9.462. Title

The title of an entity. See specific usage.

9.9.463. TLS

The Transport Layer Security (TLS) attributes.

9.9.464. To

The email header To values, as defined by RFC 5322.

9.9.465. Traffic

The network traffic refers to the amount of data moving across a network at a given point of time. Intended to be used alongside Network Connection.

9.9.466. Transaction UID

The unique identifier of the transaction.

9.9.467. Transmission Time

The event transmission time from one device to another. See specific usage

9.9.468. Tree UID

The tree id is a unique SMB identifier which represents an open connection to a share.

9.9.469. TTL

The time interval that the resource record may be cached. Zero value means that the resource record can only be used for the transaction in progress, and should not be cached.

9.9.470. Type

The type of an object or value, normalized to the caption of the type_id value. In the case of 'Other', it is defined by the event source. See specific usage.

9.9.471. Type ID

The normalized type identifier of an object. See specific usage.

Table 24 — Type ID enumerable values

Value	Name	Description
99	Other	The type is not mapped. See the type attribute, which may contain a data source specific value.
0	Unknown	The type is unknown.

9.9.472. Type Name

The event type name, as defined by the type_uid.

9.9.473. Type ID

The event type ID. It identifies the event's semantics and structure. The value is calculated by the logging system as: class_uid * 100 + activity_id.

9.9.474. Types

The type/s of an entity. See specific usage.

9.9.475. Unique ID

The unique identifier. See specific usage.

9.9.476. Alternate ID

The alternate unique identifier. See specific usage.

9.9.477. Unmapped Data

The attributes that are not mapped to the event schema. The names and values of those attributes are specific to the event source.

9.9.478. URL

The URL object that pertains to the event or object. See specific usage.

9.9.479. URL String

The URL string. See RFC 1738. For example: http://www.example.com/download/trouble.exe.

9.9.480. User

The user that pertains to the event or object.

9.9.481. HTTP User-Agent

The request header that identifies the operating system and web browser.

9.9.482. User Result

The result of the user account change. It should contain the new values of the changed attributes.

9.9.483. UUID

The universally unique identifier. See specific usage.

9.9.484. Value

The value that pertains to the object. See specific usage.

9.9.485. Vector String

The CVSS vector string is a text representation of a set of CVSS metrics. It is commonly used to record or transfer CVSS metric information in a concise form. For example: 3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:H.

9.9.486. Vendor Name

The name of the vendor. See specific usage.

9.9.487. Version

The version that pertains to the event or object. See specific usage.

9.9.488. VLAN

The Virtual LAN identifier.

9.9.489. VPC UID

The unique identifier of the Virtual Private Cloud (VPC).

9.9.490. Vulnerabilities

This object describes vulnerabilities reported in a security finding.

9.9.491. Vulnerability

The vulnerability object describes details related to the observed vulnerability

9.9.492. Web Resources

Describes details about web resources that were affected by an activity/event.

9.9.493. Web Resources Result

The results of the activity on web resources. It should contain the new values of the changed attributes of the web resources.

9.9.494. X-Forwarded-For

The X-Forwarded-For header identifying the originating IP address(es) of a client connecting to a web server through an HTTP proxy or a load balancer.

9.9.495. X-Originating-IP

The X-Originating-IP header identifying the emails originating IP address(es).

9.9.496. Extended Attributes

An unordered collection of zero or more name/value pairs where each pair represents a file or folder extended attribute.For example: Windows alternate data stream attributes (ADS stream name, ADS size, etc.), user-defined or application-defined attributes, ACL, owner, primary group, etc. Examples from DCS: ads_nameads_sizedaclownerprimary_grouplink_name - name of the link associated to the file.hard_link_count - the number of links that are associated to the file.

9.9.497. Network Zone

The network zone or LAN segment.

9.9.498. Condition

The rule trigger condition for the rule. For example: SQL_INJECTION.

9.9.499. Sensitivity

The sensitivity of the firewall rule in the matched event. For example: HIGH.

9.9.500. Match Location

The location of the matched data in the source which resulted in the triggered firewall rule. For example: HEADER.

9.9.501. Match Details

The data in a request that rule matched. For example: '["10","and","1"]'.

9.9.502. Rate Limit

The rate limit for a rate-based rule.

9.10. Objects in the ICD Schema

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

10. Events

Events are represented by instances of Event Classes, which are particular sets of attributes and objects representing a log line or telemetry submission at a point in time. Event classes have semantics that describe what happened: either a particular activity, disposition or both.

It is the intent of the schema to allow for the mapping of any raw event to a single event class. This is achieved by careful design using composition rather than a multiple inheritance approach. In order to completely capture the information in a rich data source, many attributes may be required.

Unfortunately, not every data source emits the same information for the same observed behavior. In the interest of consistency, accuracy and precision, the schema event classes specify which dictionary attributes are essential, (recommended or required), while others are optional as not all are needed across different data sources. Attribute requirements, aside from Classification attributes from the Base Event class, are always within the scope of the event class definition and not tied to the attributes themselves.

By convention, all event classes extend the Base Event event class. Attributes of the Base Event class can be present in any event class and are termed Base Attributes.

10.1. Base Event Class Attributes

The Base Event class has required, recommended, and optional attributes that apply to all core schema classes. The required attributes must be populated for every core schema event. Optional Base Event class attributes may be included in any event class, along with event class-specific optional attributes. Individual event classes will include their own required and recommended attributes.

- Examples of required base attributes are class_uid, category_uid, activity_id, severity_id.
- Examples of recommended base attributes are timezone_offset, status_id, message.
- Examples of optional base attributes are activity_name, start_time, end_time, count, duration, unmapped.
- Each event class has a unique class_uid attribute value which is the event class identifier. It is a required attribute whose value overrides the nominal Base Event class value of 99. Event class friendly names are defined by the schema, optionally populate the class_name attribute and are descriptive of the specific class, such as File System Activity or Process Activity.
- Every event class has a category_uid attribute value which indicates which is Category the class belongs to. An event class may be of only one category. Category friendly names are defined by the schema, optionally populate the category_name attribute and are descriptive of the specific category the class belongs to, such as System Activity or Network Activity.
- Every event class has an activity_id Enum attribute, constrained to the values appropriate for each event class. The semantics of the class are further defined by the activity_id attribute, such as Open for File System Activity or Launch for Process Activity. By convention, activity_id Enum labels are present tense imperatives. The Enum label optionally may populate the activity_name attribute, which is a sibling to the activity_id Enum attribute but as a Classification group attribute, follows the _name suffix convention.

10.2. Special Base Attributes

There are a few base attributes that are worth calling out specifically. These are the unmapped attribute, the raw_data attribute and the type_uid attribute.

While most if not all fields from a raw event can be parsed and tokenized, not all are mapped to the schema. The fields that are not mapped may be included with the event in the optional unmapped attribute.

The raw_data optional attribute holds the event data as received from the source. It is unparsed and represented as a String type.

The type_uid required attribute is constructed by the combination of the event class of the event (class_uid)and its activity (activity_id). It is unique across the schema hence it has a _uid suffix. The type_uid friendly name, type_name, is a way of identifying the event in a more readable and complete way. It too is a combination of the names of the two component parts.

The value is calculated as: class_uid * 100 + activity_id.

For example:

type_uid = 3001 * 100 + 1 = 300101

type_name = "Authentication: Logon"

A snippet of a File Activity event example with random values is shown below9:

```
{
"activity_id": 11,
"activity_name": "Decrypt",
"actor": {},
"category_name": "System Activity",
"category_uid": 1,
"class_name": "File System Activity",
"class_uid": 1001,
"device": {},
"end_time": 1685403212867803,
"file": {},
"message": "entry queue amateur",
"metadata": {},
"observables": [],
"severity": "Low",
"severity_id": 2,
"start_time": 1685403212792508,
"status": "img logs grove",
"status_detail": "barrier filled clothes",
"time": 1685403212834047,
"type_name": "File System Activity: Decrypt",
"type_uid": 100111
}
```

Figure 4

10.3. Constraints

As discussed in a previous section, an event class can have constraints that are more versatile than simple Required attribute requirements. When at least one of a set of recommended attributes must be present, the class can assert the at_least_one constraint:

```
 "constraints": {
   "at_least_one": [
     "ip",
     "mac",
     "name",
     "hostname"
   ]
 }
Or the just_one constraint:

"constraints": {
   "just_one": [
     "privileges",
     "group"
   ]
 }
}
```

Figure 5

10.4. Associations

Attributes within an event class are sometimes associated with each other and in some cases only one of them is present in the event while another may be looked up at processing or storage time. The schema denotes this within a class definition via the association construct:

```
"associations": {
   "actor.user": [
     "src_endpoint"
   ],
   "dst_endpoint": [
     "user"
   ],
   "src_endpoint": [
     "actor.user"
   ],
   "user": [
     "dst_endpoint"
   ]
 }
```

Figure 6

In this example from the Authentication class, the user as actor associates with its endpoint, the src_endpoint attribute of the class, while the target user associates with its endpoint, the dst_endpoint of the class. Note that the associations in this class are bi-directional, which is common, although uni-directional associations are also possible in other situations.

The construct may be useful for automated processing systems where a lookup service is available for an attribute that isn't or can't be populated via the source event producer. In these cases the processor_time should be populated at the time of the association, as with other types of enrichments.

10.5. Events in the ICD Schema

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

====

Description

11. Categories

A Category organizes event classes that represent a particular domain. For example, a category can include event classes for different kinds of events that may be found in an access log, or audit log, or network and system events. Each category has a unique category_uid attribute value which is the category identifier. Category IDs also have category_name friendly name attributes, such as System Activity, Network Activity, Audit, etc.

An example of categories with some of their event classes is shown in the below table.

Table 25 — Example of categories with some of their event classes

System Activity	Network Activity	Identity Access Management	Findings	Discovery	Application Activity
File System Activity	Network Activity	Account Change	Security Finding	Device Inventory Info	Web Resources Activity
Kernel Extension Activity	HTTP Activity	Authentication		Device Config State	Application Lifecycle
Kernel Activity	DNS Activity	Authorize Session			API Activity
Memory Activity	DHCP Activity	Entity Management			Web Resrouce Access Activity
Module Activity	RDP Activity	User Access Management			
Scheduled Job Activity	SMB Activity	Group Management			
Process Activity	SSH Activity				
	FTP Activity				
	Email Activity				

System Activity	Network Activity	Identity Access Management	Findings	Discovery	Application Activity
	Network File Activity				
	Email File Activity				
	Email URL Activity				

Finding the right granularity of categories is an important modeling topic. Categorization is weakly structural while event classification is strongly structural (i.e. it defines the particular attributes, their requirements, and specific Enum values for the event class).

Many events produced in a cloud platform can be classified as network activity. Similarly, many host system events include network activity. The key question to ask is, do the logs from these services and hosts provide the same context or information? Would there be a family of event classes that make sense in a single category? For example, does the NLB Access log provide context/info similar to a Flow log? Does network traffic from a host provide similar information to a firewall or router? Are they structured in the same fashion? Do they share attributes? Would we obscure the meaning of these logs if we normalize them under the same category? Would the resultant category make sense on its own or will it lose its contextual meaning all together?

Using profiles, some of these overlapping categorical scenarios can be handled without new partially redundant event classes.

11.1. Categories for ICD Schema

The ICD Schemas categories organize event classes, each aligned with a specific domain or area of focus.

11.1.1. System Activity

System Activity events.

11.1.2. Findings

Findings events report findings, detections, and possible resolutions of malware, anomalies, or other actions performed by security products.

11.1.3. Identity & Access Management

Identity & Access Management (IAM) events relate to the supervision of the system's authentication and access control model. Examples of such events are the success or failure of authentication, granting of authority, password change, entity change, privileged use etc.

11.1.4. Network Activity

Network Activity events.

11.1.5. Discovery

Discovery events report the existence and state of devices, files, configurations, processes, registry keys, and other objects.

11.1.6. Application Activity

Application Activity events report detailed information about the behavior of applications and services.

12. Profiles

Profiles are overlays on event classes and objects, effectively a dynamic mix-in class of attributes with their requirements and constraints. While event classes specialize their category domain, a profile can augment existing event classes with a set of attributes independent of category. Attributes that must or may occur in any event class are members of the Base Event class. Attributes that are specialized for selected classes are members of a profile.

Multiple profiles can be added to an event class via an array of profile values in the optional profiles attribute of the Base Event class. This mix-in approach allows for reuse of event classes vs. creating new classes one by one that include the same attributes. Event classes and instances of events that support the profile can be filtered via the profiles attribute across all categories and event classes, forming another dimension of classification.

For example, a Security Controls profile that adds MITRE ATT&CKTM Attack and Malware objects to System Activity classes avoids having to recreate a new event class, or many classes, with all of the same attributes as the System Activity classes. A query for events of the class will return all the events, with or without the security information, while a query for just the profile will return events across all event classes that support the Malware profile. A Host profile can add Device, and Actor objects to Network Activity event classes when the network activity log source is a user's computer. Note that the Actor object includes Process and User objects, so a Host profile can include all of these when applied. A Cloud profile could mix-in cloud platform specific information onto Network Activity events.

The profiles attribute is an optional array attribute of the Base Event class. The absence of the profiles attribute means no profile attributes are added as would be expected. Attributes defined with a profile have requirements that cannot be overridden, since profiles are themselves optional; it is assumed that the application of a profile is because those attributes are desired and can be populated.

However some classes, such as System Activity classes, build-in the attributes of a profile, for example the Host profile attributes device and actor are defined in the class. When a class definition includes the profile attributes, it still registers for that profile in the class definition so as to match any searches across events for that profile. In this case the class defined attribute requirement definitions take precedence.

Core schema profiles for Security Control, Host, Cloud, Container and Linux (for the Linux extension described later) are shown in the below table with their attributes.

Table 26 — Core schema profiles

Security Control	Host	Cloud	Container	Linux
attacks	actor	api	container	group
disposition_id / disposition	device	cloud	namespace_pid	euid
malware				egid
				auid

A special Date/Time profile adds Datetime typed time attributes in every class where there is a Timestamp time attribute. This allows for human readable RFC-3339 strings paired with epoch UTC integer values.

Other profiles could be product oriented, such as Firewall, IDS, VA, DLP etc. if they need to add attributes to existing classes. They can also be more general, platform oriented, such as for Mac, Linux or Windows environments.

The core schema comes with a Linux profile via the Linux platform extension.

Vendors can add profiles via extensions. For example, Splunk Technical Add-ons might define a profile that could be added to all events with Splunk's standard source, sourcetype, host attributes.

12.1. Disposition

The disposition_id attribute of the Security Control profile indicates the outcome or state of the event class' activity at the time of event capture and is an Enum with a standard set of values, such as Blocked, Quarantined, Deleted, Delayed.

Only event classes that register for the profile may have a disposition_id but all have an activity_id. A typical use of disposition_id is when a security protection product detects a threat and blocks it. The activity might have been a file open, but if the file was infected, the disposition would be that the file open was blocked. As of this writing, disposition_id is added to core schema classes only via the Security Controls profile.

12.2. Profile Application Examples

Using example categories and event classes from a preceding section, examples of how profiles might be applied to event classes are shown below.

12.2.1. System Activity

The event classes would all include the Host profile and may include the Security Controls or Cloud profile.

12.2.2. Network Activity

The event classes may include the Host profile and may include the Security Controls or Cloud profile.

12.2.3. Identity & Access Management

The event classes would include the Host profile, (due to actor.user), **may** include the Cloud profile, and would not include the Security Control profile.

12.3. Personas and Profiles

The personas called out in an earlier section, producer, author, mapper, analyst, all can consider the profile from a different perspective.

Producers, who can also be authors, can add profiles to their events when the events will include the additional information the profile adds. For example a vendor may have certain system attributes that are added via an extension profile. A network vendor that can detect malware would apply the Security Controls profile to their events. An endpoint security vendor can apply the Host, User and Security Controls profile to network events.

Authors define profiles, and the profiles are applicable to specific classes, objects or categories.

Mappers can add the profile ID and associated attributes to specific events mapped to logs in much the same way producers would apply profiles.

Analysts, e.g. end users, can use the browser to select applicable profiles at the class level. They can use the profile identifier in queries for hunting, and can use the profile identifiers for analytics and reporting. For example, show all malware alerts across any category and class.

12.4. Profiles

==== Profile:

Description

==== Profile:

Description

==== Profile:

Description

==== Profile:

Description

==== Profile:

Description

==== Profile:

Description

==== Profile:

Description

==== Profile:

Description

13. Extensions

The schema can be extended by adding new attributes, objects, categories, profiles and event classes. A schema is the aggregation of core schema entities and extensions.

Extensions allow a particular vendor or customer to create a new schema or augment an existing schema.10 Extensions can also be used to factor out non-essential schema domains keeping a schema small. Extensions to the core schema use the framework in the same way as a new schema, optionally creating categories, profiles or event classes from the dictionary. Extensions can add new attributes to the dictionary, including new objects. Extended attribute names can be the same as core schema names but this is not a good practice for a number of reasons. As with categories, event classes and profiles, extensions have unique IDs within the framework as well as versioning.11

As of this writing, two platform extensions augment the core schema: Linux and Windows. The Linux extension adds a profile, while the Windows extension adds three classes to the System Activity category.

Another use of extensions to the core schema is the development of new schema artifacts, which later may be promoted into the core schema or to a platform extension. Another use of extensions is to add vendor specific extensions in addition to the core schema. In this case, a best practice is to prefix the schema artifacts with a short identifier associated with the extension range registered.12 Lastly, as mentioned above, entirely new schemas can be constructed as extensions.

Examples of new experimental categories, new event classes that contain some new attributes and objects are shown in the table below with a Dev extension superscript convention. In the example, extension classes were added to the core Findings category, and three extension categories were added, Policy, Remediation and Diagnostic, with extension classes.

Table 27 — Examples of new experimental categories, new event classes and objects

Findings	PolicyDev	RemediationDev	DiagnosticDev

Incident CreationDev	Clipbaord Content ProtectionDev	File RemediationDev	CPU UsageDev
Incident AssociateDev	ComplianceDev	Folder RemediationDev	Memory UsageDev
Incident ClosureDev	Compliance ScanDev	Startup Application RemediationDev	ThroughputDev
Incident UpdateDev	Content ProtectionDev	User Session RemediationDev	
Email Delivery FindingDev	Information ProtectionDev		

Annex A

Guidelines for attribute names

(This annex forms an integral part of this Recommendation.)

- Attribute names must be a valid UTF-8 sequence.
- Attribute names must be all lower case.
- Combine words using underscore.
- No special characters except underscore.
- Reserved attributes are prefixed with an underscore.
- Use present tense unless the attribute describes historical information.
- activity_id enum labels should be present tense. For example, Delete. disposition_id enum labels should be past tense. For example, Blocked.
- Use singular and plural names properly to reflect the attribute content.
- For example, use events_per_sec rather than event_per_sec.
- When an attribute represents multiple entities, the attribute name should be pluralized and the value type should be an array.
  - Example: process.loaded_modules includes multiple values — a loaded module names list.
- Avoid repetition of words where possible.
  - Example: device.device_ip should be device.ip.
- Avoid abbreviations when possible.
- Some exceptions can be made for well-accepted abbreviations. Example: ip, or os.
- For vendor extensions to the dictionary, prefix attribute names with a 3-letter moniker in order to avoid name collisions. Example: aws_finding, spk_context_ids.

Annex B

Data Types

(This annex forms an integral part of this Recommendation.)

Refer to https://schema.ocsf.io/data_types for the OCSF data types and their validation constraints.

Annex C

Schema Construction and Extension

(This annex forms an integral part of this Recommendation.)

The OCSF schema repository can be found at https://github.com/ocsf/ocsf-schema.

The repository is structured as follows:

Table C.1 — Repository structure

File or Folder	Purpose
categories.json	the schema categories are defined and must be present for classes to be in a category
dictionary.json	the schema dictionary is where all attributes must be defined
version.json	the schema semver version, every change to the schema requires this file be updated
enums/	the schema enum definitions, optional if enums are shared
events/	the schema event classes
extensions/	the schema extensions, a similar structure is set per extension
includes/	the schema shared files
objects/	the schema object definitions
profiles/	the schema profiles

For information and examples about how to add to the schema, see CONTRIBUTING.md in the OCSF GitHub.

Extending the Schema To extend the schema create a new directory using a unique extension name (e.g. dev) in the extensions directory. The directory structure is the same as the top level repository structure above, and it may contain the following files and subdirectories, depending on what type of extension is desired:

Table C.2 — Repository structure

File or Folder	Purpose
categories.json	Create to define a new event category to reserve a range of class IDs
dictionary.json	Create to define new attributes
events/	Create to define new event classes
objects/	Create to define new objects
profiles/	Create to define new profiles

In order to reserve an ID space, and make your extension public, add a UID to your extension name in the Extensions Registry here to avoid collisions with core or other extension schemas. For example, the dev extension would have a row in the table as follows:

Table C.3 — Dev extension example

Extension Name	Type	UID	Notes
Development	dev	999	The development schema extensions

New categories and event classes will have their unique IDs offset by the UID.

More information about extending existing schema artifacts can be found at extending-existing-class.md.

Bibliography